

贪心算法 & 动态规划

主讲：黄凤林

➔ 贪心算法基本概念

所谓贪心算法是指，在对问题求解时，总是**做出在当前看来是最好的选择**。也就是说，不从整体最优上加以考虑，他所做出的仅是在某种意义上的局部最优解。

贪心算法没有固定的算法框架，**算法设计的关键是贪心策略的选择**。必须注意的是，贪心算法不是对所有问题都能得到整体最优解，选择的贪心策略必须具备无后效性，即某个状态以后的过程不会影响以前的状态，只与当前状态有关。

所以对所采用的贪心策略一定要仔细分析其是否满足无后效性。

➔ 贪心算法的基本思路

- ① 建立数学模型来描述问题。
- ② 把求解的问题分成若干个子问题。
- ③ 对每一子问题求解，得到子问题的局部最优解。
- ④ 把子问题的解局部最优解合成原解问题的一个解。

➡ 贪心算法适用的问题

- ▶ 贪心策略适用的前提是：
- ▶ **局部最优策略能导致产生全局最优解。**
- ▶ 实际上，贪心算法适用的情况很少。一般，对一个问题分析是否适用于贪心算法，可以先选择该问题下的几个实际数据进行分析，就可做出判断。一定要注意判断问题是否适合采用贪心算法策略，找到的解是否一定是问题的最优解，要想用贪心算法必须严格证明且没有反例。

➡ 贪心算法---例题1

给定一个 $n*m$ 的矩阵，求解矩阵从第一行到 n 行，每行且选一个数，使其和最大。

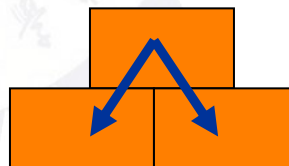
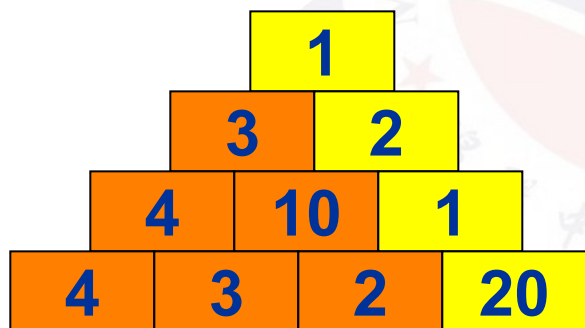
问题分析：

- ✓ 要使其整体和最大，务必每行的值最大。
- ✓ 因此只需要找出每行的最大值然后求其和即可。

➔ 贪心算法---例题2

给定一个数字三角形(如下图)

- 从第一行的数开始, 每次可以向左下或右下走一格, 直到最后一行
 - 要求一路上所有数的和最大
- 行数: $N \leq 100$



➡ 例题2---问题分析

- 根据问题分析
- 按照例题1的思考方式每次向下走，选择值最大的方向向下走。
- 这样子的贪心策略是否正确呢？
- **答案肯定是不正确的，见例图。**
- 那如何做才是正确的呢？
- 我们反过来想，从上往下走，也就相当于下面的数来自于上面两个方向。
- **假设下面位置为 $[i,j]$ ，这他的值来自于 $[i-1,j-1]$ 和 $[i-1,j]$ 两个方向中交大的那个数的值。**
- **而最后的答案可以是最下面一行任意位置的值构成的路径，所以我们还需要取一个max。**
- **所以我们用 $dp[i][j]$ 表示从第一行开始走到 $[i,j]$ 这个位置构成路径的最大值。则： $dp[i][j] = \max(dp[i-1][j-1], dp[i-1][j])$ 。**
- **最后 $ans = \max(dp[n][j]), 1 \leq j \leq n$ 。**

这也就是我们后面要讲的动态规划

➡ 贪心算法---例题3

➤ 背包问题

有一个背包，背包容量是 $M=150$ 。有7个物品，每个物品分别有一个体积和价值，物品可以分割成任意大小。

要求尽可能让装入背包中的物品总价值和最大，但不能超过总容量。

✓ 物品	A	B	C	D	E	F	G
✓ 重量	35	30	60	50	40	10	25
✓ 价值	10	40	30	50	35	40	30

➡ 例题3---问题分析

- 贪心策略1：每次挑选价值最大的物品装入背包，得到的结果是否最优？
- 贪心策略2：每次挑选所占重量最小的物品装入是否能得到最优解？
- 贪心策略3：每次选取单位重量价值最大的物品，成为解本题的策略。

➡ 贪心算法适用范围

- 值得注意的是，贪心算法并不是完全不可以使用，贪心策略一旦经过证明成立后，它就是一种高效的算法。
- 贪心算法是很常见的算法之一，这是由于它简单易行，构造贪心策略不是很困难。可惜的是，它需要证明后才能真正运用到题目的算法中。
- 一般来说，贪心算法的证明围绕着：整个问题的最优解一定由在贪心策略中存在的子问题的最优解得来的，也就是局部最优使得整体最优。

➡ 例题：排队打水

- 有 N 个人排队接水。每个人接水所用的时间不同，而一个时刻只能有一个人接水，且只有等一个人接完水后，另一个人才能开始接水。现在请你求出 N 个人接水的顺序，使得每个人所用平均时间最短，并输出这个最短平均时间。(洛谷 P1233)
- 数据范围
 - 对于20%的数据， $N \leq 20$
 - 对于60%的数据， $N \leq 500$
 - 对于100%的数据， $N \leq 1000000$

➡ 排队打水---问题分析

- 对于20%的数据，想都不用想就知道是搜索剪枝。
- 对于后面的数据，想都不用想就知道搜索过不了
- 我们知道深搜慢的原因在于重复了很多次，比如同样一个序列abc，如果他前面序列不同或者后面序列不同，我们都需要重新搜索一次。
- 但是实际上我们完全可以把它存储起来，下次直接调用，所以我们可以考虑记忆化搜索或者dp，但是记录状态很麻烦...状压DP,而且效率很低，500都过不了。
- 怎么办呢？ 此时我们考虑贪心

➡ 排队打水---问题分析

- 如果我们根据生活中的实例思考一下就会发现，**打水顺序不影响打水时间，影响的只是等待的时间，如果前面的人打水越快，那么后面的人等待时间就越短。**
- 比如两个人打水时间为 a 和 b ($a > b$)
- 先 a 后 b 的顺序：则所用总共时间为 $a + (a + b)$
- 先 b 后 a 的顺序：则所用总共时间为 $b + (b + a)$
- 那么明显可以看出先 b 后 a 效率更高，即让打水时间短的先打水。所以一遍sort就搞定了

➡ 例题：接水问题

学校里有一个水房，水房里一共装有 m 个龙头可供同学们打开水，每个龙头每秒钟的供水量相等，均为1。

现在有 n 名同学准备接水，他们的初始接水顺序已经确定。将这些同学按接水顺序从1到 n 编号， i 号同学的接水量为 w_i 。接水开始时，1到 m 号同学各占一个水龙头，并同时打开水龙头接水。当其中某名同学 j 完成其接水量要求 w_j 后，下一名排队等候接水的同学 k 马上接替 j 同学的位置开始接水。这个换人的过程是瞬间完成的，且没有任何水的浪费。即 j 同学第 x 秒结束时完成接水，则 k 同学第 $x+1$ 秒立刻开始接水。若当前接水人数 n 不足 m ，则只有 n 个龙头供水，其它 $m-n$ 个龙头关闭。

现在给出 n 名同学的接水量，按照上述接水规则，问所有同学都接完水需要多少秒。

➡ 接水问题—问题分析

- 很明显，这是一道水题，因为序列是固定的，所以每次把下一个人放到先结束的那个水龙头就可以了，然后每次找接水时间最短的水龙头，可以考虑优先队列优化。
- 但是没必要，因为数据很小。

NOI

➡ 例题：种树

- 一条街的一边有几座房子。因为环保原因居民想要在路边种些树。路边的地区被分割成块，并被编号成 $1..N$ 。每个部分为一个单位尺寸大小并最多可种一棵树。每个居民想在门前种些树并指定了三个号码 B , E , T 。这三个数表示该居民想在 B 和 E 之间最少种 T 棵树。当然， $B \leq E$ ，居民必须记住在指定区不能种多于区域地块数的树，所以 $T \leq E - B + 1$ 。居民们想种树的各自区域可以交叉。你的任务是求出能满足所有要求的最少的树的数量。

➡ 种树---问题分析

- 根据题意来看，很容易想到，把数种在交叉部分这样能让种的数最少，看上去好像是区间问题，需要数据结构来解，但是实际上完全不需要。按照终点排序，终点相同按照起点排序，这样就能保证重合的区间是有序的。如果有重合肯定是从上一段的末尾开始重合的，所以我们就把树从末尾开始往前种，但是还有个细节需要注意，自己下来思考

➡ 例题：删数问题

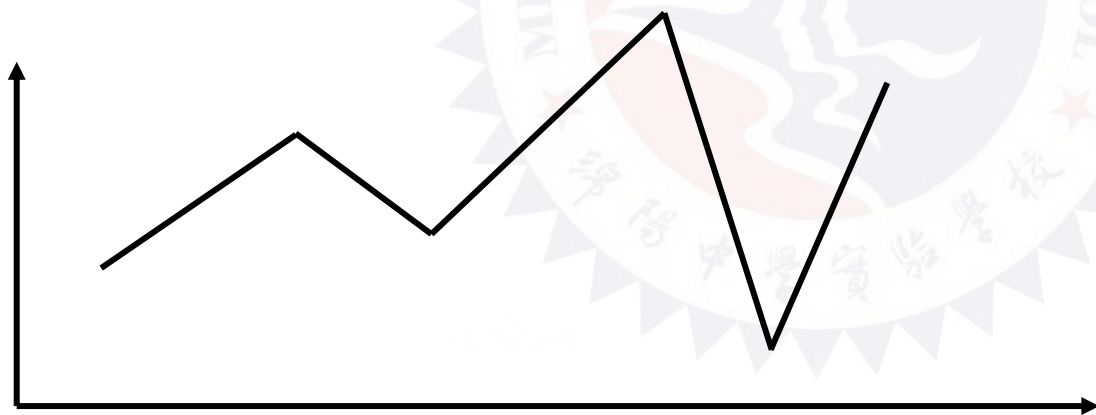
- 键盘输入一个高精度的正整数N，去掉其中任意k个数字后剩下的数字按原左右次序将组成一个新的正整数。编程对给定的N和k，寻找一种方案使得剩下的数字组成的新数最小。
- 输出应包括所去掉的数字的位置和组成的新的整数。
(N不超过250位) 输入数据均不需判错。

样例输入：
175438
4

样例输出：
13

➡ 删数问题---问题分析

- 看了题很容易想到是贪心策略，先删除最大的数，但是实际上并不是这样，随便举出一个反例。
- 比如：1415 删4明显比删5要好
- 该题也是贪心，但是和你们想象的裸贪不太一样，你可以多举几个例子来看下规律。



我们可以用折线图表示一个数的每一位。

➡ 删输问题---问题分析

- 画图之和就可以发现，我们每次先删除靠前的山峰会让剩下的值最小，即删除第一个 $a[i] > a[j]$ 的点或者最后一个点(可能单调不递减)
- **这是一道普及-的题，但是完全可以修改一下数据范围，修改为 $n \leq 100w$ 位，增加一下难度。**
- **思考下，这样又应该如何做？**

➡ 例题---军训站队

班上总共有 n 位同学，他们站成一排，然而，本该站成一条直线的他们却排成了“一字长蛇阵”。用数学的语言来描述，如果把训练场看成一个平面直角坐标系，第 i 名同学所在位置的横坐标是 i ，而所有同学的纵坐标本该是 0 （或者任意一个相等的常量），这样就排成了一条直线。当然，由于同学们排的歪歪扭扭，所以第 i 位同学的横坐标依然是 i ，而纵坐标却成了 Y_i （为了简化问题，我们假设所有的坐标都是整数）。对此，教官当然十分生气，因此他决定下命令调整队伍，使得所有人能够站成一条直线（也即让所有的 Y_i 相同）。教官的命令总共有三种：

- 1、除了某一个同学外，其余所有同学向前走一步（向前走一步可理解为 Y_i 的值加 1，下同）；
 - 2、除了某一个同学外，其余所有同学向前走两步；
 - 3、除了某一个同学外，其余所有同学向前走五步。
- 问最少需要多少次命令使得所有同学站成一条直线

➡ 军训战队---问题分析

- 本题有一个很重要的思想转变就是，一个人不动，其他人向前移动；等价于其他人不动，一个人向后退。
- 于是问题变成了所有前面的人向后退，退到最后一个人的最少时间。



➡ 例题：电池的寿命

- 小S新买了一个掌上游戏机，这个游戏机由两节5号电池供电。为了保证能够长时间玩游戏，他买了很多5号电池，这些电池的生产商不同，质量也有差异，因而使用寿命也有所不同，有的能使用5个小时，有的可能就只能使用3个小时。显然如果他只有两个电池一个能用5小时一个能用3小时，那么他只能玩3个小时的游戏，有一个电池剩下的电量无法使用，但是如果他有更多的电池，就可以更加充分地利用它们，比如 he 有三个电池分别能用3、3、5小时， he 可以先使用两节能用3个小时的电池，使用半个小时后再把其中一个换成能用5个小时的电池，两个半小时后再把剩下的一节电池换成刚才换下的电池（那个电池还能用2.5个小时），这样总共就可以使用5.5个小时，没有一点浪费。
- 现在已知电池的数量和电池能够使用的时间，请你找一种方案使得使用时间尽可能的长。

➡ 贪心算法练习题

- 洛谷 P1250 种树
- 洛谷 P1031 均分纸牌
- 洛谷 P1094 纪念品分组
- 洛谷 P1106 删数问题
- Poj 3468 电池的寿命
- 洛谷 P1016 旅行家的预算
- 洛谷 P1325 雷达安装
- POJ 3614 sunscreen
- POJ 3190 Stall Reservations
- 洛谷 P2082 区间覆盖
- 洛谷 P1230 智力大冲浪

➡ 贪心算法分析与升级

- 贪心问题的思考方法：
 - 1、选择相邻两个位置进行比较，然后得出贪心策略。
 - 2、需要用一定的数据结构去维护其贪心内容。
 - 3、贪心常常和其他算法相结合，比如和二分答案相结合，把求解性问题变成判定性问题。
 - 4、与任务调度相关联的若干问题
 - 5、按位枚举，求出字典序最小的解

➡ 打工小题目

- 有 n 个工作，每个工作用 (a_i, b_i) 表示，令 E 为你的能力值，最开始为0。
- 做完第 i 个工作，能得到 $E \cdot a_i$ 的钱，在得到钱后你的能力值会加上 b_i 。
- 问如何安排你的工作顺序，使得能得到最多的钱，为多少？

➡ 打工小题目---问题分析

- 加速相邻两个任务为 i, j , 在完成前 $i-1$ 个任务的能力为 E 。
- 先做 i 再做 j 得到的钱为: $E * a[i] + (E + b[i]) * a[j]$
- 先做 j 在做 i 得到的钱为: $E * a[j] + (E + b[j]) * a[i]$
- 则两则做减法得到 $b[i] * a[j] - b[j] * a[i]$, 要使得 i 先做比 j 先做更优, 则 $b[i] * a[j] - b[j] * a[i] > 0$, 整理得到
- $b[i] / a[i] > b[j] / a[j]$ 更优。因此按照 $b[i] / a[i]$ 从大到小排序贪心即可。

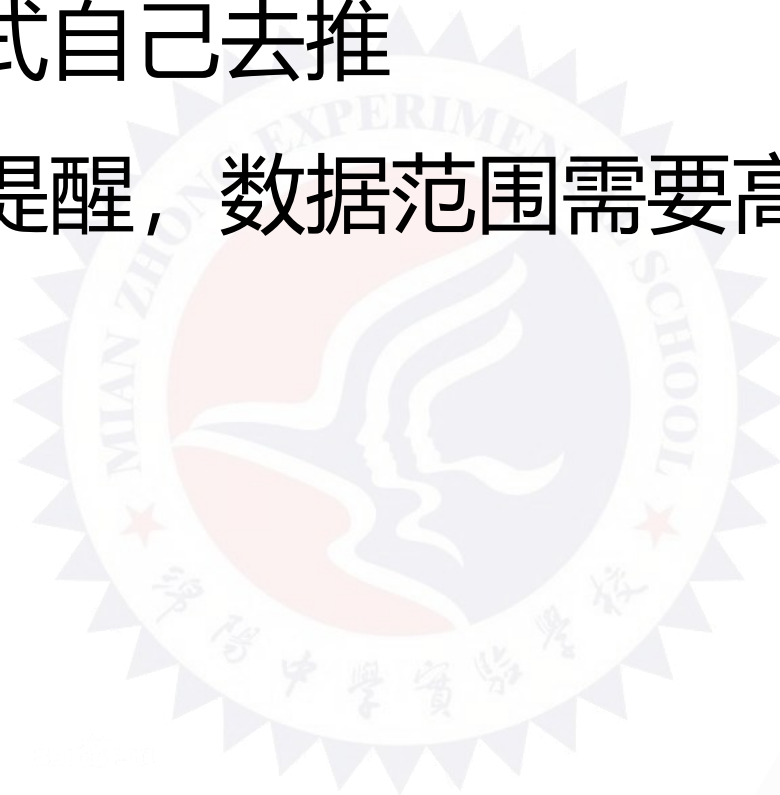
➡ 例题：国王的游戏

- 恰逢H国国庆，国王邀请 n 位大臣来玩一个有奖游戏。首先，他让每个大臣在左、右手上面分别写下一个整数，国王自己也在左、右手上各写一个整数。然后，让这 n 位大臣排成一排，国王站在队伍的最前面。排好队后，所有的大臣都会获得国王奖赏的若干金币，每位大臣获得的金币数分别是：排在该大臣前面的所有人的左手上的数的乘积除以他自己右手上的数，然后向下取整得到的结果。国王不希望某一个大臣获得特别多的奖赏，所以他想请你帮他重新安排一下队伍的顺序，使得获得奖赏最多的大臣，所获奖赏尽可能的少。
- 注意，国王的位置始终在队伍的最前面。
- 对于60%的数据，有 $1 \leq n \leq 100$ ；对于60%的数据，保证答案不超过 10^9 ；
对于100%的数据，有 $1 \leq n \leq 1,000$ ， $0 < a、b < 10000$ 。
- **NOIP 2012 提高组**

→ 国王的游戏---问题分析

贪心公式自己去推

只是想提醒，数据范围需要高精度



➡ 例题：旅行家的预算

- 一个旅行家想驾驶汽车以最少的费用从一个城市到另一个城市（假设出发时油箱是空的）。给定两个城市之间的距离 $D1$ 、汽车油箱的容量 C （以升为单位）、每升汽油能行驶的距离 $D2$ 、出发点每升汽油价格 P 和沿途油站数 N （ N 可以为零），油站 i 离出发点的距离 D_i 、每升汽油价格 P_i （ $i=1, 2, \dots, N$ ）。计算结果四舍五入至小数点后两位。如果无法到达目的地，则输出“No Solution”。
- $N \leq 10^5$

样例输入

275.6 11.9 27.4 2.8 2

102.0 2.9

220.0 2.2

样例输出

26.95

➔ 旅行家的预算---问题分析

- 先考虑无解的情况，很简单，自己脑补。
- 接着考虑有解的情况，如果按照题面意思考虑很复杂，情况很多，很难处理。因此我们考虑到每一个加油站都无脑把油箱先加满，然后到了看油箱里面是否有比当前加油站贵的油，然后把贵的退回去的思路。具体做法：
 - 1、在第一个加油站把油加满
 - 2、接着考虑每一个加油站，每个加油站先把邮箱加满，接着假设邮箱里有V的油比当前加油站贵，就把这些油退回去，替换成这个加油站的油。
 - 3、开车的时候优先用便宜的油于是我们就可以用一个**双端队列**来实现

旅行家的预算---举例

➤ 300 20 12 2 3

➤ 50 1.9

➤ 140 2.2

➤ 240 1.8



➡ 例题：跳石头

- 一年一度的“跳石头”比赛又要开始了！这项比赛将在一条笔直的河道中进行，河道中分布着一些巨大岩石。
- 组委会已经选择好了两块岩石作为比赛起点和终点。在起点和终点之间，有 n 块岩石（不含起点和终点的岩石）。在比赛过程中，选手们将从起点出发，每一步跳向相邻的岩石，直至到达终点。
- 为了提高比赛难度，组委会计划移走一些岩石，使得选手们在比赛过程中的**最短跳跃距离尽可能长**。
- 由于预算限制，组委会至多从起点和终点之间移走 m 块岩石（不能移走起点和终点的岩石）。(noip2015)

➡ 跳石头---问题分析

- 这是一道的经典二分答案题目，有二分答案很显然的标志：最小值最大，所以我们可以把这样一个求解问题变成一个判断性问题，我们假定最短的跳跃距离为 K 。
- 思考该如何验证？
- 要想每两个石头之间的最短距离为 K 即相邻两个石头之间的距离必须大于等于 K ，一开始很多相邻石头间的距离是小于 K 的，因此必须要移走一些石头才能满足。
- 验证于是变成了：为满足相邻石头间的距离 $\geq k$ ，则必须要移走的最少石头，如果移走的石头数量小于等于给定的移走石头数量 M ，则证明可以，否则证明不行。

➡ 例题：Alyona and mex

- 对于数列A，令 $A[L,R]$ 是一个数组
- $\text{mex}(A)$ 表示：在A数组中没出现过的最小非负整数，例如： $\text{mex}([0,1,2,3])=4$ ； $\text{mex}([0,1,2,5])=3$
- 给定m个区间 $[l_i, r_i]$ ，找到一个长度为n的数列A，使得满足m个区间的 $\min(A[l_1, r_1], A[l_2, r_2], \dots)$ 的值最大，并求出最大值，数列A不唯一，满足要求即可。
- codeforces739A.

→ Mex---问题分析

- 我们这样子考虑
- 长度为n的数组A, $\text{mex}(A)$ 的最大值为多少? 为n
- 因此 $\min(A[l_1, r_1], A[l_2, r_2] \dots)$ 的最大值为 $\min(r_i - l_i + 1)$
- 因此我们令 $k = (r_i - l_i + 1)$, 则我们只需要构造
- $\min(A[l_1, r_1], A[l_2, r_2] \dots) = k$ 的数列
- **则 $A[i] = i \% k$; 即可**

➡ 例题： 活动安排

- 有 n 个活动需要使用教室，每个活动都有对应的起止时间 $[s_i, f_i]$ ，问如何安排才能举办更多的活动，最多能举办的活动有多少？（MSOJ: 1498）

输入：

4

1 3

4 6

2 5

1 7

输出：

2

活动安排---问题分析

- **这是活动安排问题中最基本的一个问题，如何在时间不冲突的情况下选择更多的活动。**
- 根据贪心的思想，很明显开始时间最早的我们可以优先考虑，所以我们可以先把任务按照开始时间排序。
- 但是很容易想到并不是开始时间最早的就一定要去完成，比如 $s_i < s_j$ 且 $f_i > f_j$ 。
- 这种情况下我们就可以用j活动去替换i活动，终止时提前了，这样就腾出了更多的时间，但是活动数量并不会发生改变。
- 如果 $s_i < s_j$ 且 $f_i \leq f_j$ ，i比j更优，就放弃j
- 如果 $s_j > f_i$ ，说明两个任务不相交，那么后面的也肯定和i不相交了，所以就确定了i，先暂定j。

→ 活动安排---参考代码

```
for(int i=1;i<=n;i++)
{
    if(a[i].s>=now)//开始时间最接近的两个任务不相交
    {
        num++;
        now=a[i].f;
    }
    else if(a[i].f<now)//下一个任务比当前选中的任务先结束，那么换一个
    now=a[i].f;
}
```



➡ 活动安排---问题分析2

- 前一种解题思路是先在不冲突的情况下暂定一个最近的活动，如果后面有活动比当前选中的活动更优，那么就替换掉当前活动。在比较的过程中我们会发现，起作用的是终止时间，即终止时间越早的就是我们需要的。所以我们可以按照终止时间排序。
- 我们很容易想到，终止时间最早的任务一定是必选的，因为没有任何任务可以替换他，并且腾出更多的时间。同样的道理，对于下一个终止时间最早的任务，只要和上一个已经选定的任务不冲突，那么该任务也一定是必选的(和前一种思路不一样的地方在于，该方法只要选中了就是必选，而前一种方法是暂定)。

➡ 例题：人员安排

- 已知一个活动的起止时间，需要安排人员值班，每个人员都有固定的上下班时间(左闭右开)，问最少安排多少人才才能保证从活动开始到结束都有人值班。
- $N \leq 100000$; MSOJ 1499

输入：

3 10

1 7

3 6

6 10

输出：

2

➡ 问题分析

- 该题题意是需要最少的人把整个活动完全覆盖。
- 首先，我们肯定先选择开始上班时间最早的工作人员，只有这样才能保证从一开始就完全覆盖。所以我们需要按照起点排序。
- 其次，在当前选择的情况下，我们选择的下一个人是要和我们当前区间相交(相切)的，如果可以选择的人有多个呢？我们要如何取舍呢？
- 既然是要求人员最少，那么很明显，如果这个满足条件的人工作时间越晚(终止时间越晚)，这样我们需要的人就会相对更少。



➔ 问题分析

- 该题的解题思路相对比较简单，但是对代码的要求有一定难度。
- 对于当前已经选中的值班人员*i*，我们要去后面找一个和*i*的值班时间有交集并且下班时间最晚的那一个，来作为下一次当做参考的值班人员。

```
int Max=0;
for(int j=i+1;j<=n;j++)
{
    if(a[j].s>a[now].f)//没有交集
        break;
    if(a[j].f>=a[now].f+1&&a[j].f>a[Max].f)
        //从有交集中选择一个终止时间最晚的
        Max=j;
}
if(Max==0) i++;
else
{
    i=now=Max;
    cnt++;
}
```

**注意起点相同时
终点的排序顺序**

➡ 任务调度相关类型

- 有 n 个任务一台机器，每个任务ddl(截止的时间)为 d_i ，时间为 t_i 。如果一个任务在 $f_i(f_i > d_i)$ 完成，那么它的延迟为 $f_i - d_i$ ，求任务安排方式使得最大延迟最小。



➡ 例题：活动安排问题

- 有若干个活动，第 i 个活动开始时间和结束时间是 $[s_i, f_i)$ ，同一个教室安排的活动之间不能交叠，求要安排所有活动，最少需要几个教室。（ $n \leq 100000$ ）
- 51Nod 1428:活动安排问题

输入：

3

1 2

3 4

2 9

输出：

2

➡ 例题：区间覆盖

- 已知有N个区间，每个区间的范围是 $[si, ti]$ 。
- 请求出区间覆盖后的总长。
- 落谷 P2082 区间覆盖

输入：

3

1 100000

200001 1000000

100000000 100000001

输出：

900002

→ 区间覆盖---问题分析

- 我们按照线段的左端点排序，然后依次处理每条线段。
- 例如：第一条线段 $L1, R1$ ，第二条线段为 $L2, R2$
- 如果 $L2 \leq R1$ ，则两条线段覆盖的的长度为 $R2 - L1 + 1$ ，
- 如果 $L2 > R1$ ，则线段覆盖的总长度为：
- $(R1 - L1 + 1) + (R2 - L2 + 1)$



➡ 例题：Main Sequence

- 用一个长度为 n 的 “(” 或者 “)” 序列，问是否可以把其中的某些 “(” 变为 “)” 保证该式子括号匹配(只能把 “(” 变成 “)” ，不能反过来)。
- Codeforces 286C: Main Sequence

输入：

3

((((

()))

)(((

输出：

Yes

No

NO

➡ Main Sequence---问题分析

- 该题也比较简单，因为我们只能把“(”变成“)”，所以如果我们正着枚举，因为无法判断变还是不变，所以很麻烦，所以我们干脆倒着枚举，枚举到一个“)”，那肯定需要一个“(”来匹配，如果枚举到一个“(”，并且后面没有与之匹配的“)”，很明显，此处的“(”肯定要变成“)”，最后看下是否还有剩下的“(”或者“)”就可以了，和普通字符串模拟一样，用一个栈模拟就可以了。

➡ 例题：随机数生成器

- 一个 $n*m$ 的矩阵，上面填着1到 $n*m$ 的整数排列
- 求一条从 $(1,1)$ 到 (n,m) 的路径，每一步只能向右或向下走。
- **路径的序列为路径上数字排序之后的序列**，求出权值字典序最小的路径。
- $n, m \leq 5000$ 。

2	4	6
1	3	5
9	8	7

权值字典序最小的路径为： 1 2 3 5 7

-

➡ 例题：线段覆盖2

- 数轴上有 n 条线段，线段的两端都是整数坐标，每条线段有一个价值，请从 n 条线段中挑出若干条线段，使得这些线段两两不覆盖(端点可以重合)且线段价值之和最大。
- $N \leq 1000$;
- CODEVS 3027 线段覆盖2

输入：

3

1 2 1

2 3 2

1 3 4

输出：

4

➡ 贪心算法---升级练习题

- 洛谷 P1315 观光公交
- 洛谷 P1550 打井
- 洛谷 P1607 庙会班车
- 洛谷 P2218 覆盖问题
- 洛谷 P2279 消防局的设立

➡ 贪心算法---扩展

- 部分贪心：
- 百度搜索：2008年国家集训队部分贪心。



The background features a central point from which several wide, colorful rays emanate, creating a starburst effect. The rays are in shades of blue, orange, red, and grey. Faint, semi-transparent binary code (0s and 1s) is scattered across the background, particularly on the right side. A small line graph with two fluctuating lines, one orange and one blue, is also visible on the right side.

Thank you!